Variables in C

A variable is the name of the memory location. It is used to store information. Its value can be altered and reused several times. It is a way to represent memory location through symbols so that it can be easily identified.

Variables are key building elements of the C programming language used to store and modify data in computer programs. A variable is a designated memory region that stores a specified data type value. Each variable has a ***unique identifier***, its ***name***, and a ***data type*** describing the type of data it may hold.

Syntax:

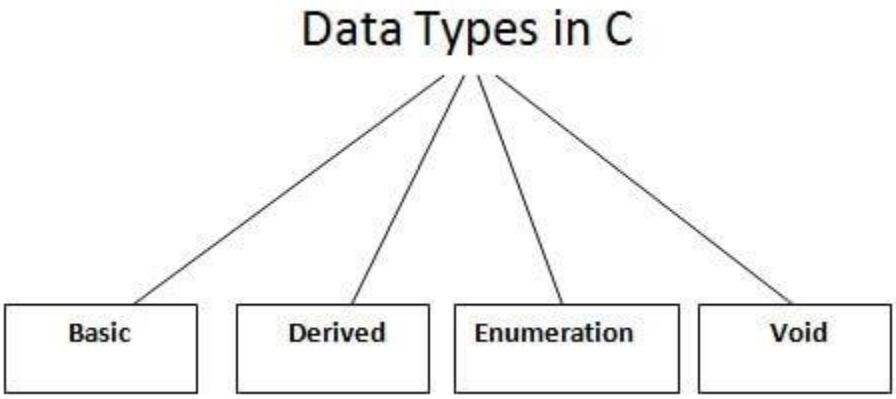
The syntax for defining a variable in C is as follows:

data_type variable_name;

Ex - int age;

Data Types in C

A data type specifies the type of data that a variable can store such as integer, floating, character, etc.



There are the following data types in C language.

Types	Data Types
Basic Data Type	int, char, float, double
Derived Data Type	array, pointer, structure, union

Enumeration Data Type	enum
Void Data Type	void

Basic Data Types

The basic data types are integer-based and floating-point based. C language supports both signed and unsigned literals.

The memory size of the basic data types may change according to 32 or 64-bit operating system.

Let's see the basic data types. Its size is given **according to 32-bit architecture**.

Data Types	Memory Size	Range
char	1 byte	−128 to 127
signed char	1 byte	−128 to 127
unsigned char	1 byte	0 to 255
short	2 byte	−32,768 to 32,767
signed short	2 byte	−32,768 to 32,767
unsigned short	2 byte	0 to 65,535
int	2 byte	−32,768 to 32,767
signed int	2 byte	−32,768 to 32,767
unsigned int	2 byte	0 to 65,535
short int	2 byte	−32,768 to 32,767
signed short int	2 byte	−32,768 to 32,767

unsigned short int	2 byte	0 to 65,535
long int	4 byte	-2,147,483,648 to 2,147,483,647
signed long int	4 byte	-2,147,483,648 to 2,147,483,647
unsigned long int	4 byte	0 to 4,294,967,295
float	4 byte	
double	8 byte	
long double	10 byte	

C Identifiers

C identifiers represent the name in the C program, for example, variables, functions, arrays, structures, unions, labels, etc. An identifier can be composed of letters such as uppercase, lowercase letters, underscore, digits, but the starting letter should be either an alphabet or an underscore.

Rules for constructing C identifiers

- The first character of an identifier should be either an alphabet or an underscore, and then it can be followed by any of the character, digit, or underscore.
- It should not begin with any numerical digit.
- In identifiers, both uppercase and lowercase letters are distinct. Therefore, we can say that identifiers are case sensitive.
- Commas or blank spaces cannot be specified within an identifier.
- Keywords cannot be represented as an identifier.
- The length of the identifiers should not be more than 31 characters.
- Identifiers should be written in such a way that it is meaningful, short, and easy to read.

Example of valid identifiers

1. total, sum, average, _m _, sum_1, etc.

Example of invalid identifiers

1. 2sum (starts with a numerical digit)
2. **int** (reserved word)
3. **char** (reserved word)
4. m+n (special character, i.e., '+')

Keywords in C

Keywords in C are ***reserved words*** that have predefined meanings and are part of the C language syntax.

A list of 32 keywords in the c language is given below:

auto	break	case	char	const	continue	default	do
double	else	enum	extern	float	for	goto	if
int	long	register	return	short	signed	sizeof	static
struct	switch	typedef	union	unsigned	void	volatile	while

printf() and scanf() in C

The printf() and scanf() functions are used for input and output in C language. Both functions are inbuilt library functions, defined in stdio.h (header file).

printf() function

The **printf() function** is used for output. It prints the given statement to the console.

The syntax of printf() function is given below:

1. printf("format string",argument_list);

Ex - printf("%d",age);

The **format string** can be %d (integer), %c (character), %s (string), %f (float) etc.

scanf() function

The **scanf() function** is used for input. It reads the input data from the console.

1. scanf("format string",argument_list);

Ex - scanf("%d",&age);

C Format Specifier

The Format specifier is a string used in the formatted input and output functions. The format string determines the format of the input and output. The format string always starts with a '%' character.

ANSI C defines a number of format specifiers. The following table lists the different specifiers and their purpose −

Format Specifier	**Type**
%c	Character
%d	Signed integer
%e or %E	Scientific notation of floats
%f	Float values
%g or %G	Similar as %e or %E
%hi	Signed integer (short)
%hu	Unsigned Integer (short)
%i	Unsigned integer
%l or %ld or %li	Long
%lf	Double
%Lf	Long double

%lu	Unsigned int or unsigned long
%lli or %lld	Long long
%llu	Unsigned long long
%o	Octal representation
%p	Pointer
%s	String
%u	Unsigned int
%x or %X	Hexadecimal representation

Escape Sequence in C

An escape sequence in C is a literal made up of more than one character put inside single quotes. Normally, a character literal consists of only a single character inside single quotes. However, the escape sequence attaches a special meaning to the character that appears after a backslash character (\).

The \ symbol causes the compiler to escape out of the string and provide meaning attached to the character following it.

Here is a list of escape sequences available in C −

Escape sequence	**Meaning**
\\	\ character
\'	' character
\"	" character
\?	? character

\a	Alert or bell
\b	Backspace
\f	Form feed
\n	Newline
\r	Carriage return
\t	Horizontal tab
\v	Vertical tab
\ooo	Octal number of one to three digits
\xhh . . .	Hexadecimal number of one or more digits

Comments in C

Comments in C language are used to provide information about lines of code. It is widely used for documenting code. There are 2 types of comments in the C language.

1.	Single Line Comments	//this is a single line comment
2.	Multi-Line Comments	/*this is a multiple line comment*/

ASCII value in C

The full form of ASCII is the American Standard Code for information interchange. It is a character encoding scheme used for electronics communication. Each character or a special character is represented by some ASCII code, and each ascii code occupies 7 bits in memory.

In C programming language, a character variable does not contain a character value itself rather the ascii value of the character variable. The ascii value represents the character variable in

numbers, and each character variable is assigned with some number range from 0 to 127. For example, the ascii value of 'A' is 65.

In the above example, we assign 'A' to the character variable whose ascii value is 65, so 65 will be stored in the character variable rather than 'A'.

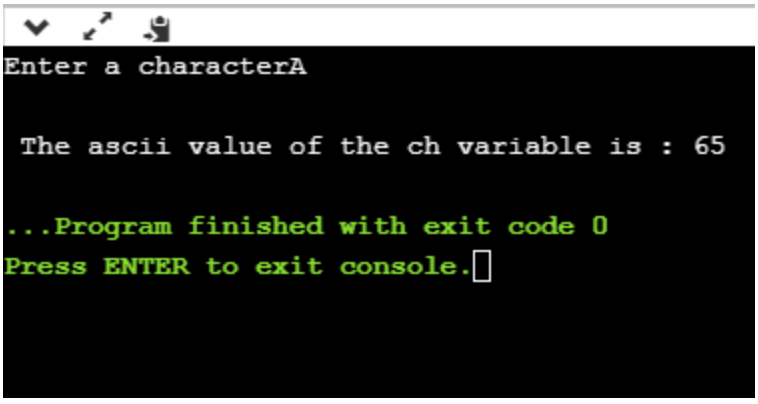
Let's understand through an example.

We will create a program which will display the ascii value of the character variable.

```
#include <stdio.h>

int main()

{

   char ch;    // variable declaration

   printf("Enter a character");

   scanf("%c",&ch);  // user input

   printf("\n The ascii value of the ch variable is : %d", ch);

   return 0;

}
```

In the above code, the first user will give the character input, and the input will get stored in the 'ch' variable. If we print the value of the 'ch' variable by using %c format specifier, then it will display 'A' because we have given the character input as 'A', and if we use the %d format specifier then its ascii value will be displayed, i.e., 65.

Output



The above output shows that the user gave the input as 'A', and after giving input, the ascii value of 'A' will get printed, i.e., 65.

Constants in C

In programming languages like *C, constants* are essential because they give you a mechanism to store *unchanging values* that hold true throughout the course of the program. These numbers may be used for several things, such as *creating mathematical constants* or *giving variables set values*.

2 ways to define constant in C

1.	const keyword	const **float** PI=3.14;
2.	#define preprocessor	#define PI 3.14

C Boolean

In C, Boolean is a data type that contains two types of values, i.e., 0 and 1. Basically, the bool type value represents two types of behavior, either true or false. Here, '0' represents false value, while '1' represents true value.

Syntax

bool variable_name;

Ex – bool x = true;

Type Casting in C

The term "type casting" refers to converting one datatype into another. It is also known as "type conversion". There are certain times when the compiler does the conversion on its own (implicit type conversion), so that the data types are compatible with each other.

On other occasions, the C compiler forcefully performs the typecasting (explicit type conversion), which is caused by the typecasting operator. For example, if you want to store a 'long' value into a simple integer then you can type cast 'long' to 'int'.

You can use the typecasting operator to explicitly convert the values from one type to another −

(type_name) expression

Example 1

Consider the following example −

```
#include <stdio.h>

int main() {

  int sum = 17, count = 5;
```

```
   double mean;
   mean =  sum / count;
   printf("Value of mean: %f\n", mean);
}
```

Output

Run the code and check its output −

```
Value of mean: 3.000000
```

While we expect the result to be 17/5, that is 3.4, it shows 3.000000, because both the operands in the division expression are of **int** type.

Example 2

In C, the result of a division operation is always in the data type with a larger byte length. Hence, we have to typecast one of the integer operands to **float**.

The cast operator causes the division of one integer variable by another to be performed as a floating-point operation −

```
#include <stdio.h>
int main() {
   int sum = 17, count = 5;
   double mean;
   mean = (double) sum / count;
   printf("Value of mean: %f\n", mean);
}
```

Output

When the above code is compiled and executed, it produces the following result −

```
Value of mean: 3.400000
```

It should be noted here that the cast operator has precedence over division, so the value of sum is first converted to type double and finally it gets divided by count yielding a double value.

Type conversions can be implicit which is performed by the compiler automatically, or it can be specified explicitly through the use of the cast operator. It is considered good programming practice to use the cast operator whenever type conversions are necessary.